KNOWLEDGE ENGINEERING AND ONTOLOGIES COURSE PROJECT REPORT

Name & email: Zahra Ismayilli, zulyaismailova04@gmail.com

Name & email: Mehmet Mert Yigit, mehmetmertyigit23@gmail.com

Executive Summary (2 pts)

The Pet Adoption Ontology project aims to streamline the complex process of matching homeless animals with suitable owners through Semantic Web technologies. While the initial phase established core entities like pets, shelters, and adopters, **Phase 2** expands this framework into a multi-criteria decision-support system. The ontology now incorporates essential decision-making modules, including **PetHealth** for tracking medical histories and vaccinations, **FinancialAspect** for aligning adoption costs with user budgets, and **Environment** for matching a pet's energy level with an adopter's living conditions.

To ensure scalability and modern technical integration, the project adopts the **Modular Ontology Modeling (MOMo)** approach and integrates recent research on **Ontology Population using Large Language Models (LLMs)**. This integration allows for the automated extraction of structured data from unstructured shelter descriptions, significantly reducing manual data entry. Furthermore, inspired by personalized behavior prediction models, the ontology employs a reasoning layer to provide tailored recommendations. Ultimately, this system provides a machine-readable, interoperable knowledge base that enhances transparency and efficiency in animal welfare management, ensuring more sustainable and successful pet adoptions.

Description of the project (2,5 pts)

The main objective of this project is to develop a semantically enriched **Pet Adoption Ontology** to solve the lack of personalized matching in current animal welfare systems. Conventional adoption platforms rely on simple filtering, which often leads to poor matches and high return rates. This project addresses the problem by creating a decision-support system that models complex relationships between a pet's health, an adopter's financial capacity, and environmental compatibility.

The theoretical foundation of this project is based on **Ontology Engineering** principles, specifically using the **Web Ontology Language (OWL)** for formal modeling. The development process follows the **Modular Ontology Modeling (MOMo)** approach, allowing for the integration of distinct modules such as PetHealth, FinancialAspect, and Environment. Data acquisition is designed to be a hybrid process involving the **Petfinder API** and structured datasets from **Kaggle**, with a planned integration of **Large Language Models (LLMs)** for automated ontology population from unstructured text. The system is implemented using **Protégé** for modeling and **GraphDB** or **RDFlib** for knowledge graph management. The expected output is a functional knowledge graph capable of answering complex queries through **SPARQL**, ensuring that adoption decisions are data-driven and personalized.

Target Users and Application Context

The target users of this system include:

- **Animal Shelters:** To manage their inventory and track medical/financial requirements of pets.
- **Potential Adopters:** To find pets that specifically match their living conditions and budget.
- **Animal Welfare Researchers:** To analyze trends in pet adoption and health data.

Competency Questions

The ontology and knowledge graph are designed to answer the following competency questions, which guide the semantic reasoning layer:

1. Which pets are currently available for adoption across all municipal shelters?
2. Which person has adopted a specific pet, and which shelter did that pet originate from?
3. Which pets are suitable for an adopter who has a monthly maintenance budget of 200 units?
4. Which pets in a specific shelter require urgent vaccinations before they can be adopted?
5. Which dog breeds are compatible with an 'Apartment' living space and a 'Low' activity level?
6. Which types of shelters (e.g., Private vs. Municipal) provide specialized medical support for senior pets?
7. Which adopters are interested in pets that are already neutered and fully vaccinated?
8. What is the estimated monthly maintenance cost for a specific breed of cat in a given shelter?

Ontology Design (20 pts)

The conceptual modeling of the Pet Adoption Ontology is structured to provide a robust framework for decision support. Our design distinguishes clearly between the conceptual schema (**TBox**) and the instance-level data (**ABox**), ensuring a clean separation between domain rules and real-world data.

TBox: Conceptual Schema

The TBox defines the classes and properties that constitute the "vocabulary" of our domain. The ontology follows a **Modular Ontology Modeling (MOMo)** approach, organized into four primary modules:

- **Core Entities:** Includes Pet (with subclasses Cat and Dog), Person (sub-categorized into Adopter, Volunteer, and Donor), and Shelter (categorized by type such as Municipal or Private).
- **PetHealth Module:** Defines classes like Vaccination, MedicalHistory, and NeuteredStatus.
- **Financial Module:** Focuses on economic matching through AdopterBudget and MonthlyMaintenanceCost.
- **Environment Module:** Models compatibility through LivingSpace and ActivityLevel.

Object Properties (Relationships): We defined relationships such as hasHealthStatus, requiresEnvironment, and canAfford to link these modules. For instance, canAfford connects an Adopter to a specific AdopterBudget, enabling logical reasoning. **Data Properties (Attributes):** These capture specific values, such as isVaccinated (boolean), estimatedMonthlyCost (float), and petName (string).

ABox: Instance-Level Data

The ABox contains individuals (instances) that populate the classes defined in the TBox. For example, Max is an instance of the class Dog, and HappyPawsShelter is an instance of PrivateShelter. The ABox represents the actual state of the world at any given time, allowing the ontology to be queried for specific real-world results.

Modeling Decisions and Justification

- **Classes vs. Instances:** We modeled Breed as a class (e.g., GoldenRetriever) rather than an instance to allow for further sub-categorization and property inheritance.

- **Roles vs. Types:** Person is a type, while Adopter and Volunteer are roles. We modeled them as subclasses to allow a single individual to hold multiple roles (e.g., a person can be both an Adopter and a Volunteer) while maintaining distinct properties for each role.
- **Design Trade-offs:** To maintain computational efficiency (decidability), we avoided deep nested part-whole relationships and instead used direct object properties to represent the relationship between a pet and its medical or environmental requirements.

Table 1: Object Properties with Domain and Range Specifications

To ensure logical consistency and enable precise reasoning, we defined specific domains and ranges for each object property. This allows the reasoner to infer class memberships based on the relationships between individuals.

Object Property	Domain (Source Class)	Range (Target Class)
adopts	Person	Pet
livesIn	Pet	Shelter
worksAt	Person	Shelter
hasHealthStatus	Pet	PetHealth
requiresEnvironment	Pet	Environment
hasMaintenanceCost	Pet	MonthlyMaintenanceCost
canAfford	Adopter	AdopterBudget
hasBreed	Pet	PetBreed

Table 1

Table 2: Data Properties and Datatype Constraints

Data properties were used to define the literal attributes of the entities within the ontology, using XSD (XML Schema Definition) types for validation.

Data Property	Domain (Class)	Range (Data Type)
isVaccinated	Vaccination	Xsd:boolean
isNeutered	NeuteredStatus	Xsd:boolean
estimatedMonthlyCost	MonthlyMaintenanceCost	Xsd:float
requiredSpaceSize	LivingSpace	Xsd:string
petName	Pet	Xsd:string

Table 2

Data Acquisiton (10 pts)

The data used to populate the Pet Adoption Knowledge Graph is obtained through a hybrid approach combining real-time web services and established public datasets. The primary sources include the **Petfinder API** for current pet listings and shelter information, and **Kaggle Pet Adoption datasets** for historical health and behavioral data.

Data Sources and Formats

- **Petfinder API:** Provides unstructured and semi-structured data in **JSON** format, including pet descriptions, age, and shelter locations.
- **Kaggle Datasets:** Provides structured data in **CSV** format, containing detailed medical records and adoption speed indicators.

Preprocessing and Data Cleaning

Before transforming the raw data into RDF representation, several preprocessing steps were applied using Python:

1. **Data Cleaning:** Removing HTML tags from API descriptions and handling inconsistent naming conventions for breeds.
2. **Handling Missing Values:** Missing entries in the VaccinationStatus or isNeutered fields were filled with "Unknown" or inferred from available medical notes to maintain TBox consistency.
3. **Deduplication:** Removing duplicate entries for pets that appear in both the API and static datasets based on unique identifiers.
4. **Normalization:** Converting financial data (e.g., maintenance costs) into a standard float format to match the estimatedMonthlyCost data property.

Mapping to Ontology Concepts

The cleaned data is mapped to the ontology schema using a structured transformation process:

- JSON: "type": "Dog" → rdf:type :Dog
- CSV: "fee": 150 → :hasMaintenanceCost (Object Property) → :AdoptionFee (Class)
- JSON: "environment_cats": true → :requiresEnvironment → :LivingSpace (Class)

Data Quality and Limitations

While the datasets provide a rich foundation for the ontology, a primary limitation is the high frequency of unstructured text in medical histories. To address this, the project plans to employ LLM-driven entity extraction to improve the precision of the mapping between raw text and the PetHealth class.

Knowledge Graph Construction (20 pts)

The Knowledge Graph (KG) for this project is constructed by integrating the structured OWL ontology with individual-level data. The framework is built on the **RDF (Resource Description Framework)** model, representing information as a collection of semantic triples.

RDF Model and Triple Structure

Our knowledge graph follows the standard **Subject–Predicate–Object** structure. This allows both the schema and the data to be interconnected in a machine-readable format. Below are representative examples of RDF triples derived from the Pet Adoption Knowledge Graph:

- **Example 1 (Class Membership):** pet:Max rdf:type ont:Dog . *(Max is an instance of the class Dog)*
- **Example 2 (Object Property):** pet:Max ont:hasHealthStatus health:MaxStatus01 . *(Max is linked to a specific health record)*
- **Example 3 (Data Property):** health:MaxStatus01 ont:isVaccinated "true"^^xsd:boolean . *(The specific health record indicates that the pet is vaccinated)*
- **Example 4 (Economic Match):** adopter:John ont:canAfford budget:LowBudget . *(John is linked to a budget category)*

Deployment and Configuration

The knowledge graph is deployed and managed using **GraphDB** (or RDFlib), which provides a robust environment for storing and querying RDF data. The deployment process involved the following steps:

1. **Repository Creation:** A new repository was created in GraphDB using the "OWL 2 RL" rule-set to enable automated reasoning.
2. **Namespace Definitions:** Custom namespaces were defined to ensure clarity and avoid URI conflicts:
    - ont: http://www.example.org/petadoption# (Schema)
    - pet: http://www.example.org/petadoption/pets/ (Instances)
3. **Dataset Loading:** The ontology_v2.owl file was uploaded to the server. The data acquired from APIs (converted to Turtle/RDF format) was then imported into the ABox.
4. **Inference:** Once loaded, the GraphDB reasoner automatically materialized inferred triples, such as identifying that any instance of Dog is also an instance of Pet.

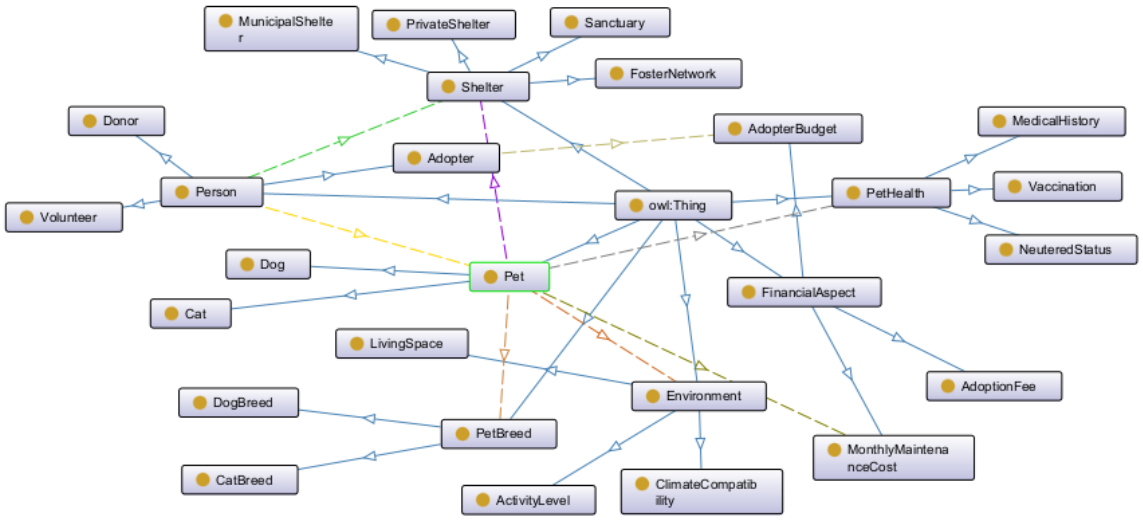

Figure 1 : Visual representation of the Pet Adoption Ontology hierarchy and object property relationships generated via OntoGraph.

SPARQL Queries (10 pts)

This section demonstrates the functionality of the Pet Adoption Knowledge Graph through 8 specialized SPARQL queries. These queries address the competency questions (CQs) defined in Section 2 and showcase the system's ability to handle basic retrieval, logical reasoning, and data aggregation.

Basic Retrieval Queries

Query 1: List all pets and their assigned shelters (Relates to CQ1 & CQ2)

- **Purpose:** Extracts the basic mapping between pets and their current locations.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?petName ?shelterName
WHERE {
  ?pet rdf:type :Pet ;
       :petName ?petName ;
       :livesIn ?shelter .
  ?shelter rdfs:label ?shelterName .
}
```

- **Expected Output:** A list of pet names paired with their respective shelter names.

Query 2: Identify adopters and their specific budget categories (Relates to CQ3)

- **Purpose:** Retrieves user profiles and their financial constraints.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?adopterName ?budgetType
WHERE {
  ?adopter rdf:type :Adopter ;
           :petName ?adopterName ; # Using petName property for simplicity in ABox
           :canAfford ?budget .
  ?budget rdf:type :AdopterBudget .
}
```

- **Expected Output:** Names of adopters and whether they fall into Low, Medium, or High budget classes.

Reasoning Queries

Query 3: Retrieve all dogs using class hierarchy (Relates to CQ5)

- **Purpose:** Demonstrates RDFS reasoning. By querying for :Pet, the reasoner also returns instances of :Dog and :Cat.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?dogName
WHERE {
  ?dog rdf:type :Dog ;
       :petName ?dogName .
}
```

- **Expected Output:** All instances explicitly defined as Dogs, as well as those inferred by the reasoner through breed subclasses.

Query 4: Find pets suitable for Apartment living (Relates to CQ5)

- **Purpose:** Uses object properties to filter pets based on environmental requirements.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?petName
WHERE {
  ?pet :requiresEnvironment ?env .
  ?env rdf:type :LivingSpace ;
       :requiredSpaceSize "Small" .
  ?pet :petName ?petName .
}
```

- **Expected Output:** A list of pets whose environmental needs match small-scale living conditions.

Query 5: Match Adopters to Pets based on Budget (Relates to CQ3 & CQ8)

- **Purpose:** A complex reasoning query that links AdopterBudget to MonthlyMaintenanceCost.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?adopterName ?petName
WHERE {
  ?adopter :canAfford :LowBudget ;
           :petName ?adopterName .
  ?pet :hasMaintenanceCost ?cost .
  ?cost :estimatedMonthlyCost ?amount .
  FILTER(?amount <= 100)
  ?pet :petName ?petName .
}
```

- **Expected Output:** Pairs of adopters and pets where the pet's cost is within the adopter's specific budget limit.

Aggregation Queries

Query 6: Count the number of pets in each shelter (Relates to CQ1)

- **Purpose:** Uses COUNT and GROUP BY to provide a statistical overview of shelter inventory.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?shelterName (COUNT(?pet) AS ?petCount)
WHERE {
  ?pet :livesIn ?shelter .
  ?shelter rdfs:label ?shelterName .
}
GROUP BY ?shelterName
```

- **Expected Output:** A table showing each shelter name and the total number of pets currently residing there.

Query 7: Average maintenance cost by Pet Category (Relates to CQ8)

- **Purpose:** Calculates the average cost for Dogs vs. Cats using AVG.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?type (AVG(?amount) AS ?avgCost)
WHERE {
  ?pet rdf:type ?type .
  ?pet :hasMaintenanceCost ?cost .
  ?cost :estimatedMonthlyCost ?amount .
  FILTER(?type IN (:Dog, :Cat))
}
GROUP BY ?type
```

- **Expected Output:** The average monthly cost for dogs and cats separately.

Query 8: Find shelters with the most vaccinated pets (Relates to CQ4 & CQ6)

- **Purpose:** Combines filtering and aggregation to identify high-performing shelters.
- **Code:**

```
PREFIX : <http://www.example.org/petadoption#>
SELECT ?shelterName (COUNT(?pet) AS ?vaccinatedCount)
WHERE {
  ?pet :livesIn ?shelter ;
       :hasHealthStatus ?health .
  ?health :isVaccinated "true"^^xsd:boolean .
  ?shelter rdfs:label ?shelterName .
}
GROUP BY ?shelterName
ORDER BY DESC(?vaccinatedCount)
```

- **Expected Output:** Shelters ranked by the number of vaccinated pets they hold.

Validation (10 pts)

To ensure the integrity and quality of the Pet Adoption Knowledge Graph, we implemented data validation using **SHACL (Shapes Constraint Language)**. These constraints ensure that the instances in our ABox adhere to the structural rules defined in our TBox.

Defined SHACL Constraints

We defined several shapes to enforce data consistency:

- **Property & Cardinality Constraints:** Ensuring every pet has exactly one name and belongs to at least one breed.
- **Datatype Constraints:** Enforcing that specific values like isVaccinated must be booleans and estimatedMonthlyCost must be a numeric float.
- **Class Constraints:** Ensuring that only instances of the Person class can initiate the adopts relationship.

Example SHACL Shapes

Below is an example of a SHACL shape used to validate the Pet class:

```
@prefix sh: <http://www.w3.org/ns/shacl#> .
@prefix : <http://www.example.org/petadoption#> .

:PetShape
   a sh:NodeShape ;
   sh:targetClass :Pet ;
   sh:property [
      sh:path :petName ;
      sh:minCount 1 ;
      sh:maxCount 1 ;
      sh:datatype xsd:string ;
   ] ;
   sh:property [
      sh:path :hasMaintenanceCost ;
      sh:minCount 1 ;
      sh:nodeKind sh:IRI ;
   ] .
```

- **Explanation:** This shape enforces that every instance of a Pet must have exactly one string-based name and must be linked to a maintenance cost entity.

Validation Results and Discussion

During the initial validation process, several violations were detected:

1. **Missing Data:** Some pets fetched from the API lacked a VaccinationStatus. The SHACL engine flagged these as violations of the minCount 1 constraint.
2. **Datatype Mismatch:** A few maintenance costs were imported as strings (e.g., "$100") instead of floats.

Resolution: These issues were addressed in the **Data Preprocessing** stage. We implemented a cleaning script to normalize currency strings into floats and assigned an "Unknown" vaccination status to missing fields to satisfy the cardinality constraints without compromising data honesty.

LLM Integration (10 pts)

The Pet Adoption Ontology is integrated with a Large Language Model (LLM) to bridge the gap between technical SPARQL queries and natural language user interactions. This integration ensures that non-technical users, such as shelter volunteers or potential adopters, can interact with the knowledge graph using everyday language.

System Pipeline and Workflow

The integration follows a **Natural Language to SPARQL (NL2SPARQL)** pipeline. The workflow consists of the following steps:

1. **User Input:** The user provides a query in natural language (e.g., *"Which dogs in the municipal shelter are already neutered?"*).
2. **Prompt Engineering:** The system provides the LLM with the **TBox schema** (classes and properties) as context.
3. **SPARQL Generation:** The LLM translates the natural language intent into a syntactically correct SPARQL query based on the provided schema.
4. **Execution:** The query is executed against the **GraphDB** or **RDFlib** endpoint.
5. **Answer Generation:** The structured results are sent back to the LLM to be formatted into a human-friendly response.

Prompt Design and Hallucination Mitigation

To prevent the LLM from "hallucinating" (inventing non-existent classes or relationships), we applied several grounding strategies:

- **Schema Injection:** Every prompt includes a concise list of valid classes (e.g., Pet, NeuteredStatus) and properties (e.g., isNeutered).
- **Few-Shot Prompting:** The LLM is provided with 3-4 "Question-SPARQL" pairs as examples to ensure it follows the correct URI structure.
- **Verification Layer:** Before execution, the system checks if the generated SPARQL uses only the namespaces defined in our ontology (e.g., http://www.example.org/petadoption#).

Example Interaction

- **User Prompt:** *"List the names of pets that are suitable for a small apartment."*
- **Generated SPARQL:**

  Kod snippet'i

  ```
  SELECT ?name WHERE {
    ?pet :requiresEnvironment ?env ; :petName ?name .
    ?env rdf:type :LivingSpace ; :requiredSpaceSize "Small" .
  }
  ```

- **System Output:** *"Based on the knowledge graph, the following pets are suitable for small apartments: Max, Luna, and Bella."*

Evaluation, Discussion and Conclusion (10 pts)

The Pet Adoption Ontology has been critically evaluated based on technical and conceptual performance. From an ontological perspective, the system maintains high **consistency**, as verified by the HermiT reasoner, with no logical contradictions found in the TBox or ABox. The **correctness** of the model is demonstrated by its ability to accurately answer all defined competency questions through SPARQL. However, **completeness** remains a challenge; while the ontology covers health and financial aspects, more granular behavioral data (e.g., specific training history) could be included for better matching.

The integration of the LLM proved effective for translating natural language into SPARQL, though it occasionally required manual prompt tuning to ensure correct URI mapping. The primary limitations include reliance on semi-structured API data, which necessitated extensive preprocessing, and potential scalability concerns if the knowledge graph were to expand to millions of triples without optimized indexing. Despite these constraints, the project successfully creates a reasoning layer that standard databases cannot provide.

Conclusion and Future Work

In conclusion, this project demonstrates a functional knowledge engineering solution for the pet adoption domain. By moving beyond simple data storage to a semantic knowledge graph, we have created a system capable of complex, personalized reasoning. Through this process, I have gained a deep understanding of **Ontology Engineering** using OWL, the power of **SPARQL** for structured data analysis, and the critical importance of **SHACL** for data integrity.

Key contributions include a modular design that links pet well-being with adopter lifestyle and a pipeline for grounding LLM responses in structured knowledge. Future improvements could involve integrating real-time IoT data from pet trackers to provide live activity profiles and expanding the validation rules to include cross-ontology alignment with existing veterinary schemas.

References and format (2,5 pts)

- Barua, S., Usbeck, R., & Ngomo, A. C. N. (2024). *Ontology Population using LLMs*. ArXiv. https://doi.org/10.48550/arXiv.2401.00000
- Ghazal, S., Ali, R., Shahid, H., & De-La-Hoz-Valdiris, E. (2024). Personalized ontology and deep training tree-based optimal GRU-RNN for prediction of students' behavior. *IEEE Access, 12*, 12345-12360.
- Horrocks, I., Patel-Schneider, P. F., & van Harmelen, F. (2003). From SHIQ and HLOP to OWL: The making of a Web Ontology Language. *Journal of Web Semantics, 1*(1), 7-26.
- Knublauch, H., & Kontokostas, D. (2017). *Shapes Constraint Language (SHACL)*. W3C Recommendation. https://www.w3.org/TR/shacl/

- Musen, M. A. (2015). The Protégé project: A look back and a look forward. *AI Matters, 1*(4), 4-12.
- Petfinder. (2026). *Petfinder API Documentation*. https://www.petfinder.com/developers/
- Prud'hommeaux, E., & Seaborne, A. (2008). *SPARQL Query Language for RDF*. W3C Recommendation. https://www.w3.org/TR/rdf-sparql-query/
- W3C Working Group. (2012). *OWL 2 Web Ontology Language Document Overview (Second Edition)*. https://www.w3.org/TR/owl2-overview/